

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Estudios Profesionales Zona Media

PRÁCTICA 16

Control Inteligente de Iluminación mediante Bluetooth

usando Raspberry Pi Pico W

Alumno: César Gael Mancilla Estrada

Clave: 382031

Carrera: Ingeniería en Mecatrónica

Materia: Microcontroladores

Profesor: Ing. Jesús Padrón

Entrega: 14 de mayo de 2026

«La automatización inteligente transforma
lo ordinario en extraordinario»

*Control de iluminación mediante Bluetooth
Raspberry Pi Pico W + Flutter + IoT*

Índice

1. Introducción	3
1.1. Contexto y Justificación	3
1.2. Importancia en la Industria	3
1.3. Objetivos	3
1.3.1. Objetivo General	3
1.3.2. Objetivos Específicos	3
2. Marco Teórico	4
2.1. Bluetooth Clásico vs Bluetooth Low Energy	4
2.2. SerialBT - Perfil SPP en Pico W	4
2.3. Arquitectura de la Aplicación Móvil (Flutter)	4
2.4. Transistor 2N2222A como Driver de Potencia	5
2.4.1. Cálculo de la resistencia de base	5
2.5. Sensor LDR (Fotorresistencia)	5
3. Desarrollo	5
3.1. Material Requerido	6
3.2. Estructura del Proyecto	6

3.3.	Arquitectura del Sistema	7
3.4.	Diagrama de Conexión	8
3.5.	Conexiones del Circuito	8
3.5.1.	Circuito de Potencia (Transistor + Relevador)	8
3.5.2.	Circuito Sensor LDR	9
3.5.3.	Conexión de Carga AC (120V)	9
3.6.	Protocolo de Comunicación Bluetooth	9
3.7.	Código Fuente del Firmware (Pico W)	10
3.8.	Código Fuente de la Aplicación Flutter	11
3.8.1.	main.dart - Punto de entrada	11
3.8.2.	bluetooth_provider.dart - Gestión Bluetooth	12
3.8.3.	home_screen.dart - Pantalla Principal con Timer	13
3.9.	Compilación y Pruebas	13
3.9.1.	Comandos de compilación del firmware	13
3.9.2.	Comandos de compilación de la app Flutter	13
3.10.	Evidencia del Circuito Físico	14
3.11.	Capturas de la Aplicación	16
3.12.	Dificultades y Soluciones	17
3.13.	Resultados Finales	17
4.	Preguntas de Repaso	18
4.1.	1. ¿Qué función cumple el transistor en el circuito?	18
4.2.	2. ¿Por qué no debe conectarse directamente el relevador a la Raspberry Pi Pico W?	18
4.3.	3. ¿Qué función tiene el diodo conectado al relevador?	18
4.4.	4. ¿Qué es una fotorresistencia?	18
4.5.	5. ¿Cómo cambia la resistencia de una LDR cuando aumenta la luz?	18
4.6.	6. ¿Qué ventajas ofrece el control inalámbrico mediante Bluetooth?	19
4.7.	7. ¿Qué diferencia existe entre el modo manual y el modo automático?	19
4.8.	8. ¿Qué ventajas tiene Flutter para el desarrollo de aplicaciones móviles?	19
4.9.	9. ¿Qué ocurre si el relevador no utiliza transistor de activación?	19
4.10.	10. ¿Qué función cumple el temporizador dentro del sistema?	20
5.	Conclusiones	20
5.1.	Resultados Obtenidos	20
5.2.	Possibles Mejoras	20
5.3.	Aplicaciones Reales del Proyecto	21
6.	Referencias	21

Introducción

Contexto y Justificación

El control inalámbrico de iluminación mediante Bluetooth representa una solución eficiente y moderna para la automatización del hogar y la industria. La Raspberry Pi Pico W, al incorporar el chip CYW43439 con soporte para Bluetooth y Wi-Fi, se presenta como una plataforma ideal para implementar sistemas de control remoto en proyectos de sistemas embebidos de bajo costo.

A través de la librería **SerialBT**, que implementa el perfil SPP (Serial Port Profile) sobre Bluetooth Clásico, es posible establecer comunicación serial inalámbrica con dispositivos móviles de forma sencilla, permitiendo el desarrollo de aplicaciones de control robustas y eficientes.

Importancia en la Industria

Los sistemas de control de iluminación inteligente son ampliamente utilizados en diversos sectores:

- **Domótica:** Automatización de hogares inteligentes
- **Eficiencia energética:** Reducción del consumo eléctrico
- **Accesibilidad:** Control remoto para personas con movilidad reducida
- **Industria 4.0:** Integración con sistemas IoT
- **Educación:** Plataforma de aprendizaje en sistemas embebidos

Objetivos

Objetivo General

Implementar un sistema de control inteligente de iluminación mediante comunicación Bluetooth utilizando la Raspberry Pi Pico W como dispositivo principal de control.

Objetivos Específicos

1. Implementar un sistema de encendido y apagado inalámbrico mediante Bluetooth utilizando la Raspberry Pi Pico W
2. Utilizar un relevador controlado mediante transistor NPN 2N2222A para manejar una carga externa de 120V AC (bombillo)
3. Diseñar una aplicación móvil en Flutter que permita controlar el sistema
4. Implementar un temporizador configurable desde la aplicación móvil con cuenta regresiva visual

5. Implementar un modo automático de activación utilizando una fotorresistencia (LDR) para detectar cambios de iluminación ambiental

Marco Teórico

Bluetooth Clásico vs Bluetooth Low Energy

Característica	Bluetooth Classic	Bluetooth LE
Canales	79 canales	40 canales
Tasa de datos	Hasta 3 Mb/s	Hasta 2 Mb/s
Potencia TX	≤ 20 dBm	≤ 10 dBm
Consumo energético	Medio-Alto	Muy bajo
Aplicaciones	Audio, transferencia	Sensores, IoT, wearables
Topologías	Punto a punto	Punto a punto, Broadcast, Malla

Tabla 1: Comparación entre Bluetooth Clásico y Bluetooth LE

SerialBT - Perfil SPP en Pico W

La librería **SerialBT** implementa el perfil SPP (Serial Port Profile) sobre Bluetooth Clásico utilizando la pila BTstack de BlueKitchen. Esta librería permite la comunicación serial inalámbrica similar a un puerto UART, facilitando el envío y recepción de comandos de texto entre el Pico W y dispositivos móviles.

Arquitectura de la Aplicación Móvil (Flutter)

Flutter es un framework de código abierto desarrollado por Google para crear aplicaciones multiplataforma desde una única base de código. Utiliza el lenguaje Dart y renderiza directamente en el canvas del dispositivo mediante el motor Skia.

Ventajas para este proyecto:

- Desarrollo rápido con Hot Reload
- Widgets Material Design 3 personalizables
- Paquete `flutter_bluetooth_serial` para comunicación SPP
- Compilación a código nativo ARM para Android
- Gestión de estado con Provider

Transistor 2N2222A como Driver de Potencia

El transistor NPN 2N2222A funciona como interruptor controlado por voltaje. Cuando la base recibe una señal HIGH (3.3V del GPIO), el transistor entra en saturación permitiendo el paso de corriente colector-emisor.

Cálculo de la resistencia de base

$$I_B = \frac{V_{GPIO} - V_{BE(sat)}}{R_B} = \frac{3,3V - 0,7V}{1000\Omega} = 2,6 \text{ mA} \quad (1)$$

$$I_{C(max)} = \beta \times I_B = 100 \times 2,6\text{mA} = 260 \text{ mA} \quad (2)$$

$$P_{R_B} = I_B^2 \times R_B = (0,0026)^2 \times 1000 = 0,0068 \text{ W} \quad (3)$$

El relevador solo requiere $\approx 75 \text{ mA}$, por lo que el transistor opera con suficiente margen de seguridad.

Sensor LDR (Fotorresistencia)

La fotorresistencia (LDR - Light Dependent Resistor) es un componente pasivo cuya resistencia varía inversamente con la intensidad de luz incidente:

Condición	Resistencia LDR	Voltaje en ADC
Luz directa	$\approx 1 \text{ k}\Omega$	$\approx 0,3 \text{ V}$ (ADC ≈ 100 -500)
Luz ambiente	$\approx 10 \text{ k}\Omega$	$\approx 1,65 \text{ V}$ (ADC ≈ 2000)
Oscuridad total	$\approx 100 \text{ k}\Omega$	$\approx 3,0 \text{ V}$ (ADC ≈ 3000 -4095)

Tabla 2: Comportamiento del LDR en divisor de voltaje

El LDR se configura en un divisor de voltaje con una resistencia fija de $10 \text{ k}\Omega$:

$$V_{out} = V_{in} \times \frac{R_{LDR}}{R_{10k} + R_{LDR}} \quad (4)$$

Desarrollo

Material Requerido

Cant.	Componente	Especificación
1	Raspberry Pi Pico W	RP2040 + CYW43439
1	Transistor NPN 2N2222A	TO-92, $\beta_{min} = 100$
1	Módulo Relevador 5V	1 canal, activo bajo
1	Diodo 1N4007	Protección flyback
1	Fotorresistencia (LDR)	5 mm, CdS
1	Resistencia 1 k Ω	$\pm 5\%$, 1/4 W, Base transistor
1	Resistencia 10 k Ω	$\pm 5\%$, 1/4 W, Pull-up + LDR
1	Protoboard	830 puntos
-	Cables Dupont	Macho-Hembra, Macho-Macho
1	Bombillo 120V AC	Carga controlada
1	Dispositivo Android	Bluetooth 4.0+

Tabla 3: Lista de materiales

Estructura del Proyecto

Archivo	Descripción
Practica16_Bluetooth_Luz.ino	Firmware principal del Pico W
main.dart	Punto de entrada de la app Flutter
bluetooth_provider.dart	Gestión de conexión Bluetooth
home_screen.dart	Pantalla principal (Manual, Timer, Auto)
AndroidManifest.xml	Permisos Android (Bluetooth, Ubicación)
pubspec.yaml	Dependencias del proyecto Flutter

Tabla 4: Estructura del proyecto

Arquitectura del Sistema

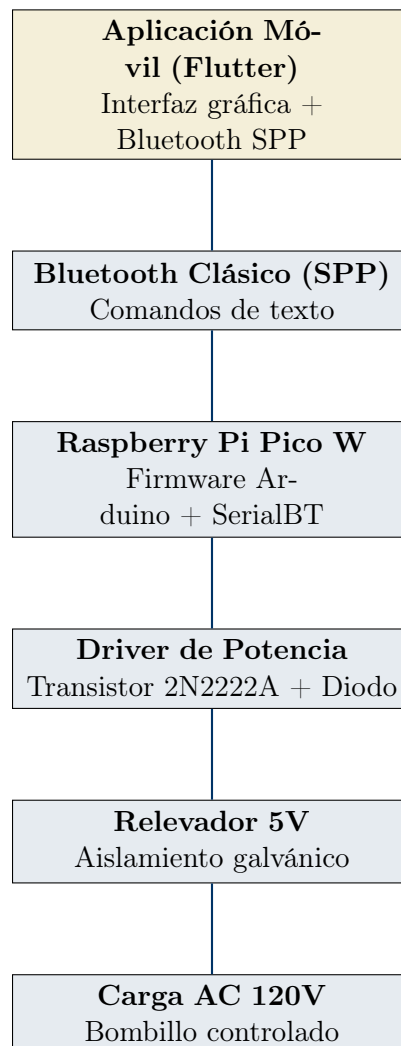


Figura 1: Arquitectura en capas del sistema

Diagrama de Conexión

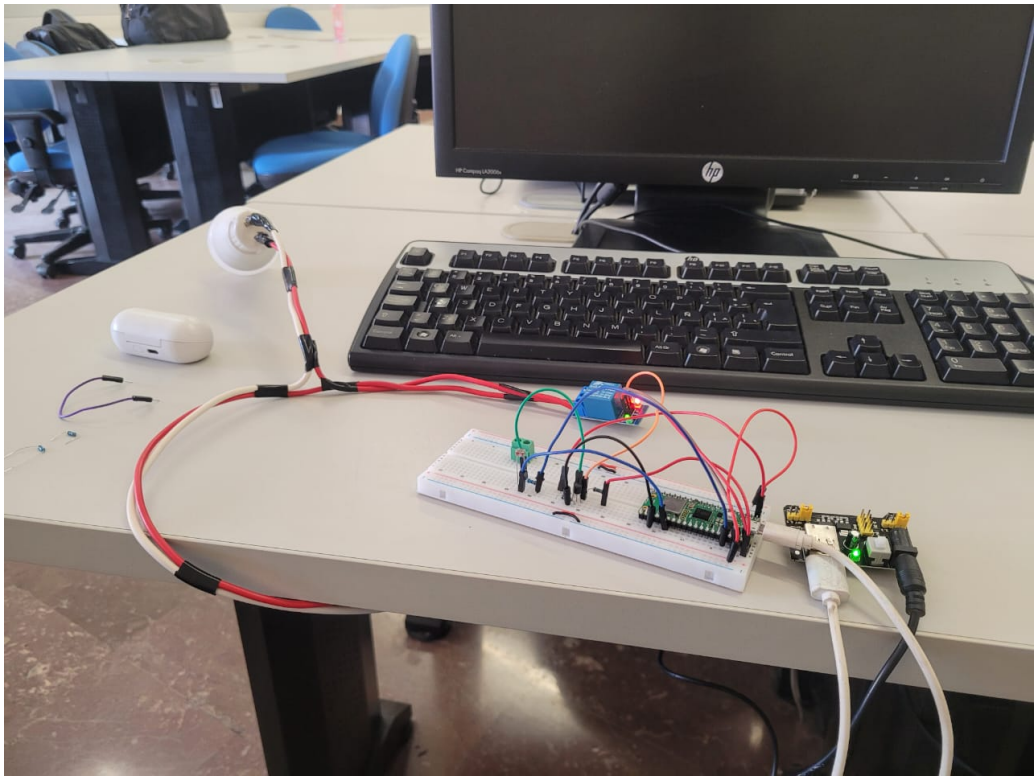


Figura 2: Diagrama de conexión del circuito completo

Conexiones del Circuito

Circuito de Potencia (Transistor + Relevador)

Pico W	Componente	Notas
VBUS (Pin 40) 5V	VCC Relevador	Alimentación
VBUS (Pin 40) 5V	10 k Ω (pull-up)	A IN del relevador
GND (Pin 38)	Emisor 2N2222A + GND Relé	Tierra común
GP0 (Pin 1)	1 k Ω $\rightarrow$ Base 2N2222A	Señal de control
-	Colector 2N2222A	IN del relevador

Tabla 5: Conexiones del circuito de potencia

Circuito Sensor LDR

Pico W	Componente	Notas
3.3V (Pin 36)	10 k Ω (extremo 1)	Alimentación divisor
GP26/Pin 31 (ADC0)	10 k Ω (ext. 2) + LDR	Punto de lectura
GND (Pin 33)	LDR (otro extremo)	Tierra

Tabla 6: Conexiones del sensor LDR (divisor de voltaje)

Conexión de Carga AC (120V)

Relevador	Conexión AC	Color típico
COM	Fase (vivo)	Negro/Rojo
NO	Bombillo $\rightarrow$ Neutro	Blanco

Tabla 7: Conexión de la carga AC

Protocolo de Comunicación Bluetooth

La comunicación entre la app y el Pico W utiliza comandos de texto plano:

Comando	Acción	Respuesta
ON	Encender bombillo	OK:ON
OFF	Apagar bombillo	OK:OFF
TIMER:30	Temporizador 30s	OK:TIMER
AUTO:ON	Modo automático LDR	OK:AUTO ON
AUTO:OFF	Modo manual	OK:AUTO OFF
STATUS	Solicitar estado	STATUS:...

Tabla 8: Protocolo de comandos Bluetooth

Código Fuente del Firmware (Pico W)

Compilación: Se utilizó Arduino CLI con la opción `ipbtstack=ipv4btcbled` para habilitar Bluetooth Clásico.

```
1 #include <Arduino.h>
2 #include <SerialBT.h>
3
4 #define RELE_PIN 0
5 #define LDR_PIN 26
6 #define LUZ_UMBRAL 2000
7
8 bool estado = false;
9 bool temporizadorActivo = false;
10 bool modoAuto = false;
11 unsigned long tiempoFin = 0;
12 String cmd = "";
13
14 void setup() {
15     pinMode(RELE_PIN, OUTPUT);
16     digitalWrite(RELE_PIN, HIGH); // Apagado (activo bajo)
17     pinMode(PIN_LED, OUTPUT);
18     digitalWrite(PIN_LED, LOW);
19     analogReadResolution(12);
20
21     for(int i=0; i<3; i++) {
22         digitalWrite(PIN_LED, HIGH); delay(200);
23         digitalWrite(PIN_LED, LOW); delay(200);
24     }
25
26     SerialBT.begin(115200);
27 }
28
29 void loop() {
30     while(SerialBT.available()) {
31         char c = SerialBT.read();
32         if(c == '\n' || c == '\r') {
33             cmd.trim();
34             cmd.toUpperCase();
35
36             if(cmd == "ON" && !modoAuto) {
37                 digitalWrite(RELE_PIN, LOW);
38                 digitalWrite(PIN_LED, HIGH);
39                 estado = true;
40                 SerialBT.println("OK:ON");
41             }
42             else if(cmd == "OFF") {
43                 digitalWrite(RELE_PIN, HIGH);
44                 digitalWrite(PIN_LED, LOW);
45                 estado = false;
46                 temporizadorActivo = false;
47                 SerialBT.println("OK:OFF");
48             }
49             else if(cmd.startsWith("TIMER:")) {
50                 int s = cmd.substring(6).toInt();
51                 if(s > 0) {
52                     tiempoFin = millis() + s * 1000;
53                     temporizadorActivo = true;
54                     digitalWrite(RELE_PIN, LOW);
55                     digitalWrite(PIN_LED, HIGH);
56                     SerialBT.println("OK:TIMER");
57                 }
58             }
59             else if(cmd == "AUTO:ON") {
60                 modoAuto = true;
61                 SerialBT.println("OK:AUTO_ON");
62             }
63             else if(cmd == "AUTO:OFF") {
64                 modoAuto = false;
65                 digitalWrite(RELE_PIN, HIGH);
66                 digitalWrite(PIN_LED, LOW);
67                 estado = false;
```

```

68         SerialBT.println("OK: AUTO_OFF");
69     }
70     cmd = "";
71 } else { cmd += c; }
72 }
73
74 if(temporizadorActivo && millis() >= tiempoFin) {
75     digitalWrite(RELE_PIN, HIGH);
76     digitalWrite(PIN_LED, LOW);
77     temporizadorActivo = false;
78     estado = false;
79     SerialBT.println("TIMER:FIN");
80 }
81
82 if(modosAuto) {
83     int luz = analogRead(LDR_PIN);
84     if(luz > LUZ_UMBRAL && !estado) {
85         digitalWrite(RELE_PIN, LOW);
86         digitalWrite(PIN_LED, HIGH);
87         estado = true;
88     }
89     else if(luz < LUZ_UMBRAL && estado) {
90         digitalWrite(RELE_PIN, HIGH);
91         digitalWrite(PIN_LED, LOW);
92         estado = false;
93     }
94 }
95 delay(50);
96 }

```

Listing 1: Firmware principal - Practica16_Bluetooth_Luz.ino

Código Fuente de la Aplicación Flutter

main.dart - Punto de entrada

```

1  import 'package:flutter/material.dart';
2  import 'package:provider/provider.dart';
3  import 'providers/bluetooth_provider.dart';
4  import 'screens/home_screen.dart';
5
6  void main() {
7      runApp(const MyApp());
8  }
9
10 class MyApp extends StatelessWidget {
11     const MyApp({super.key});
12
13     @override
14     Widget build(BuildContext context) {
15         return ChangeNotifierProvider(
16             create: (context) => BluetoothProvider(),
17             child: MaterialApp(
18                 title: 'Smart Light Control',
19                 debugShowCheckedModeBanner: false,
20                 theme: ThemeData(
21                     colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
22                     useMaterial3: true,
23                 ),
24                 home: const HomeScreen(),
25             ),
26         );
27     }
28 }

```

Listing 2: main.dart - Punto de entrada de la aplicación

bluetooth_provider.dart - Gestión Bluetooth

```

1 import 'dart:async';
2 import 'dart:convert';
3 import 'package:flutter/foundation.dart';
4 import 'package:flutter_bluetooth_serial/flutter_bluetooth_serial.dart';
5
6 class BluetoothProvider extends ChangeNotifier {
7   BluetoothState _bluetoothState = BluetoothState.UNKNOWN;
8   bool _isConnected = false;
9   BluetoothConnection? _connection;
10  BluetoothDevice? _connectedDevice;
11
12  bool _luzEncendida = false;
13  bool _modoAutomatico = false;
14  bool _temporizadorActivo = false;
15  int _tiempoRestante = 0;
16
17  List<BluetoothDevice> _devices = [];
18  bool _isScanning = false;
19
20  StreamSubscription? _discoverySubscription;
21
22  // Getters para todos los campos privados...
23
24  Future<void> startScan() async {
25    _devices.clear();
26    _isScanning = true;
27    notifyListeners();
28
29    // Obtener dispositivos vinculados
30    List<BluetoothDevice> bonded =
31      await FlutterBluetoothSerial.instance.getBondedDevices();
32    _devices.addAll(bonded);
33    notifyListeners();
34
35    // Buscar nuevos dispositivos
36    _discoverySubscription = FlutterBluetoothSerial.instance
37      .startDiscovery().listen((result) {
38        if (!_devices.any((d) => d.address == result.device.address)) {
39          _devices.add(result.device);
40          notifyListeners();
41        }
42      });
43  }
44
45  Future<bool> connectToDevice(BluetoothDevice device) async {
46    BluetoothConnection connection =
47      await BluetoothConnection.toAddress(device.address);
48    _connection = connection;
49    _isConnected = true;
50    notifyListeners();
51    return true;
52  }
53
54  Future<void> sendCommand(String command) async {
55    if (_connection?.isConnected == true) {
56      _connection!.output.add(ascii.encode('$command\r\n'));
57      await _connection!.output.allSent;
58    }
59  }
60
61  void encenderLuz() { sendCommand('ON'); _luzEncendida = true; notifyListeners(); }
62  void apagarLuz() { sendCommand('OFF'); _luzEncendida = false; notifyListeners(); }
63  void setTemporizador(int s) { sendCommand('TIMER:$s'); _temporizadorActivo = true;
64    notifyListeners(); }
65  void activarModoAuto() { sendCommand('AUTO:ON'); _modoAutomatico = true;
66    notifyListeners(); }
67  void desactivarModoAuto() { sendCommand('AUTO:OFF'); _modoAutomatico = false;
68    notifyListeners(); }
69 }

```

Listing 3: bluetooth_provider.dart - Proveedor de Bluetooth

home_screen.dart - Pantalla Principal con Timer

```

1 class _TimerTabState extends State<_TimerTab> {
2   final _min = TextEditingController();
3   final _sec = TextEditingController();
4   Timer? _countdownTimer;
5   int _localCountdown = 0;
6
7   void _start() {
8     int m = int.tryParse(_min.text) ?? 0;
9     int s = int.tryParse(_sec.text) ?? 0;
10    int total = m * 60 + s;
11    if (total > 0) {
12      context.read<BluetoothProvider>().setTemporizador(total);
13
14      // Iniciar conteo regresivo local en la app
15      setState(() { _localCountdown = total; });
16      _countdownTimer?.cancel();
17      _countdownTimer = Timer.periodic(
18        const Duration(seconds: 1),
19        (timer) {
20          if (_localCountdown > 0) {
21            setState(() { _localCountdown--; });
22          } else {
23            timer.cancel();
24          }
25        }
26      );
27      _min.clear(); _sec.clear();
28    }
29  }
30
31  String _formatTime(int seconds) {
32    int m = seconds ~/ 60;
33    int s = seconds % 60;
34    return '${m.toString().padLeft(2, '0')}:${s.toString().padLeft(2, '0')}';
35  }
36
37  // Widget build con c rculo de conteo y campos de entrada...
38 }

```

Listing 4: home_screen.dart - Fragmento del Timer con conteo regresivo

Compilación y Pruebas

Comandos de compilación del firmware

```

1 # Configurar Git en PATH
2 $env:Path = "C:\Program Files\Git\bin;C:\Program Files\Git\cmd;" + $env:Path
3
4 # Compilar con Bluetooth habilitado
5 arduino-cli compile \
6   --fqbn "rp2040:rp2040:rpipicow:ipbtstack=ipv4btcb1e" \
7   --output-dir "./build" \
8   "Practical16_Bluetooth_Luz.ino"
9
10 # Resultado:
11 # Sketch usa 451,928 bytes (21%) del espacio de programa
12 # Memoria dinámica: 89,228 bytes (34%)

```

Listing 5: Compilación del firmware Pico W

Comandos de compilación de la app Flutter

```
1 # Limpiar y compilar
2 flutter clean
3 flutter pub get
4 flutter build apk --release
5
6 # Resultado:
7 # Built build\app\outputs\flutter-apk\app-release.apk (19.9 MB)
```

Listing 6: Compilación del APK Android

Evidencia del Circuito Físico

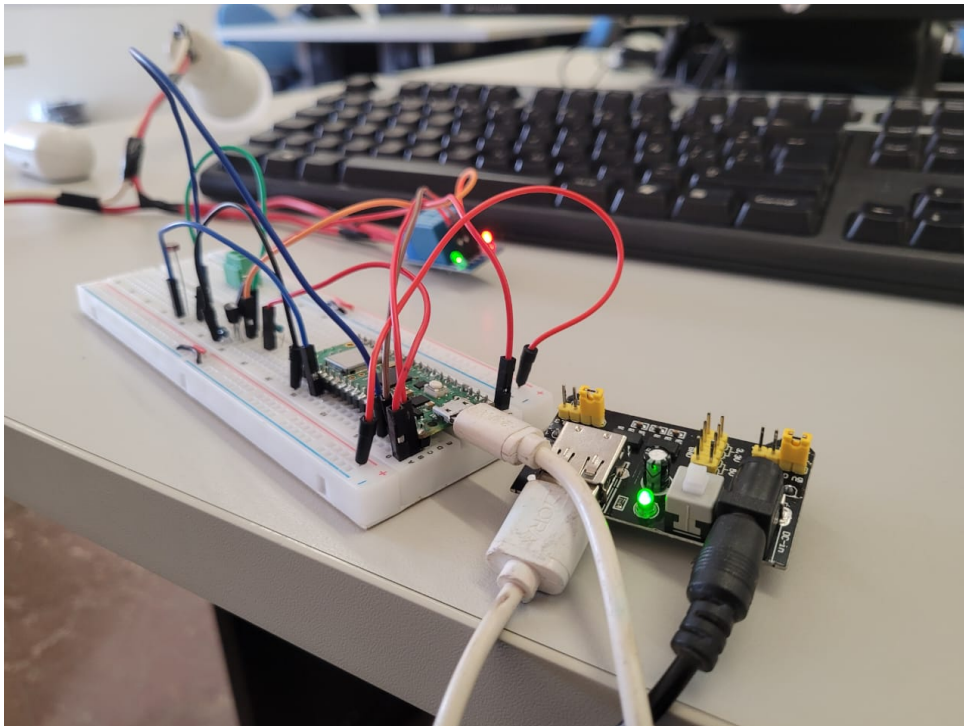
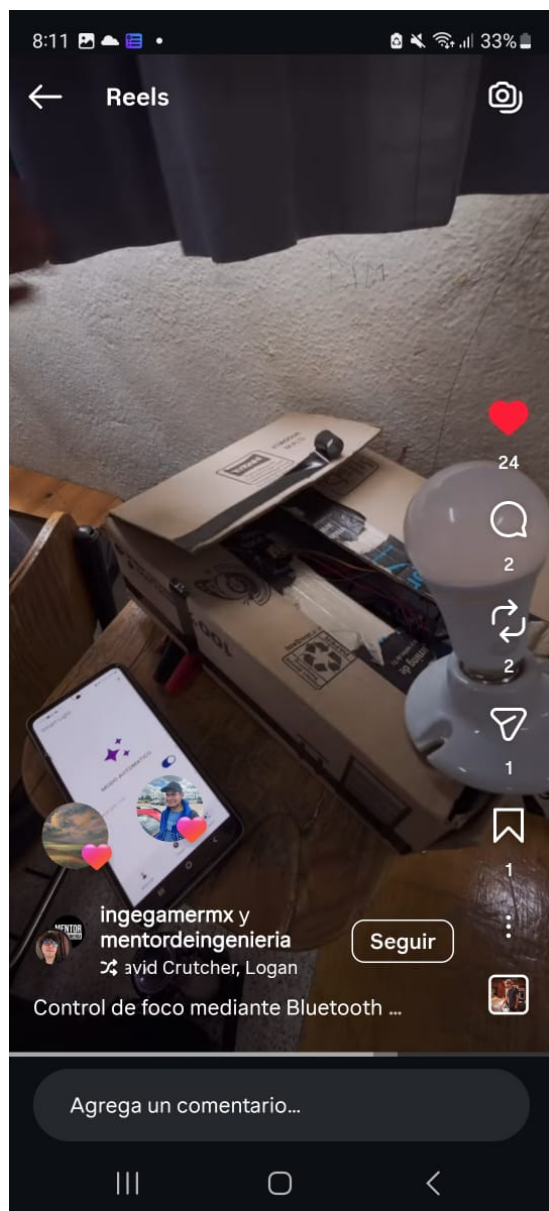
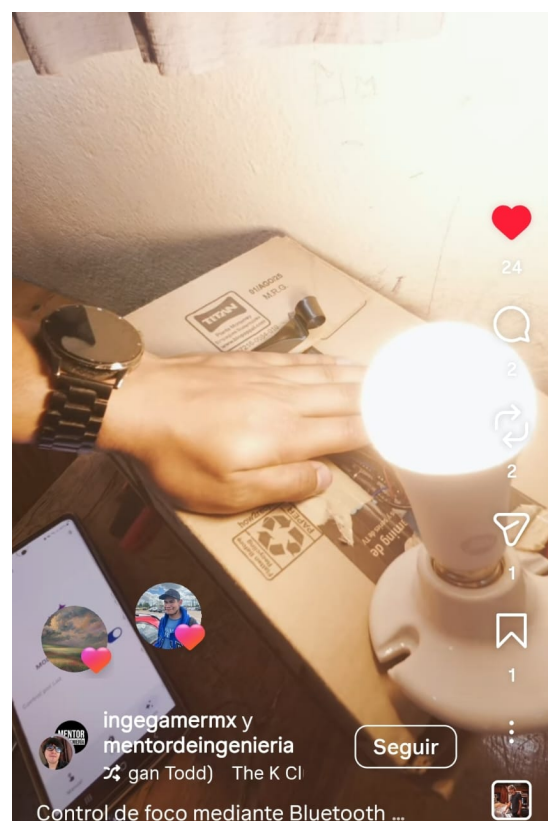


Figura 3: Circuito completo implementado en protoboard



(a) Bombillo apagado



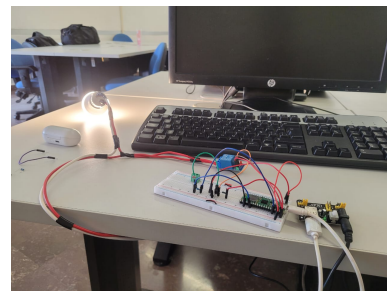
(b) Bombillo encendido

Figura 4: Evidencia del funcionamiento del bombillo 120V AC

Capturas de la Aplicación



(a) Modo Manual

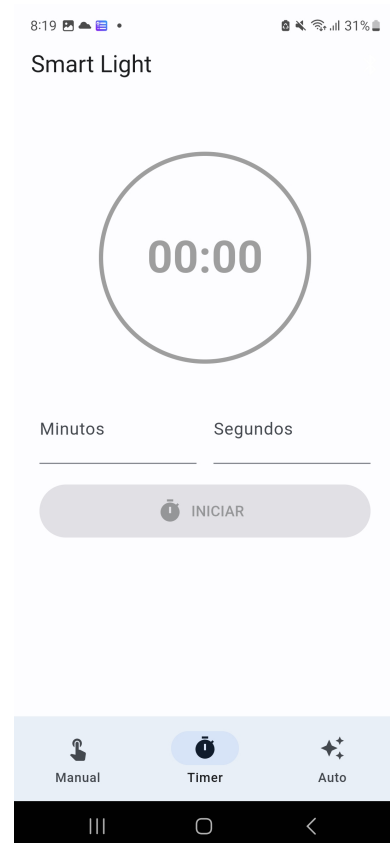


(b) Conexión Bluetooth

Figura 5: Capturas de la aplicación - Control Manual y Conexión



(a) Temporizador



(b) Modo Automático

Figura 6: Capturas de la aplicación - Temporizador y Modo Automático

Dificultades y Soluciones

Dificultad	Solución Implementada
Git no reconocido por Flutter	Instalación de Git 2.54 y configuración del PATH, eliminando conflicto con Git del Pico SDK v1.5.1
Bluetooth no se activaba en Pico W	Uso de la opción de compilación <code>ipbtstack=ipv4btcbles</code> que habilita Bluetooth Clásico + IPv4
Relevador se activaba solo al encender	Agregar resistencia pull-up de 10 kΩ entre VCC (5V) e IN del relevador
LDR funcionaba al revés	Invertir lógica de comparación: <code>luz > umbral</code> → oscuridad → encender
App no mostraba dispositivos Bluetooth	Agregar permisos <code>ACCESS_FINE_LOCATION</code> y activar GPS; llamar <code>startScan()</code> al abrir ventana
Timer sin conteo regresivo visual	Implementar <code>Timer.periodic</code> local en Flutter con actualización cada segundo
Error flutterEmbedding v1	Agregar metadato <code>flutterEmbedding:2</code> en <code>AndroidManifest.xml</code>

Tabla 9: Dificultades encontradas y soluciones implementadas

Resultados Finales

Entregable	Tamaño	Estado
<code>firmware.uf2</code>	451,928 bytes (21 % flash)	Compilado
<code>app-release.apk</code>	19.9 MB	Compilado

Tabla 10: Archivos generados

Funcionalidades verificadas:

- Encendido/Apagado manual por Bluetooth desde app propia
- Temporizador configurable con cuenta regresiva visual
- Modo automático con sensor LDR (oscuridad = encender)
- Control de carga AC 120V mediante transistor y relevador
- Comunicación Bluetooth estable entre Pico W y Android

Preguntas de Repaso

1. ¿Qué función cumple el transistor en el circuito?

El transistor 2N2222A funciona como un **interruptor controlado por voltaje y amplificador de corriente**. Cuando la base recibe una señal HIGH (3.3V) desde el pin GP0 del Pico W, el transistor entra en saturación permitiendo el paso de corriente del colector al emisor. Esto permite controlar una carga que requiere ≈ 75 mA (bobina del relevador) utilizando solo ≈ 2.6 mA desde el GPIO del microcontrolador, que solo puede suministrar 4-12 mA de forma segura.

2. ¿Por qué no debe conectarse directamente el relevador a la Raspberry Pi Pico W?

Por tres razones principales:

1. **Corriente:** Los pines GPIO del Pico W solo suministran 4-12 mA, mientras que el relevador requiere 70-100 mA para activarse
2. **Voltaje:** El Pico W opera a 3.3V en sus GPIO, insuficiente para un relevador de 5V
3. **Protección:** Sin transistor, el pico de voltaje inverso (flyback) de la bobina dañaría permanentemente el GPIO

3. ¿Qué función tiene el diodo conectado al relevador?

El diodo 1N4007 se conecta en **paralelo inverso** con la bobina del relevador (ánodo al colector/IN, cátodo a VCC). Cuando el transistor deja de conducir, el campo magnético de la bobina colapsa generando un pico de voltaje inverso que podría dañar el transistor. El diodo proporciona un camino de baja impedancia para esta corriente, disipándola de forma segura.

4. ¿Qué es una fotorresistencia?

Una fotorresistencia o LDR (Light Dependent Resistor) es un componente electrónico pasivo fabricado con materiales semiconductores como el sulfuro de cadmio (CdS). Su resistencia eléctrica varía en **función inversa** a la intensidad de luz que incide sobre ella: a mayor luz, menor resistencia.

5. ¿Cómo cambia la resistencia de una LDR cuando aumenta la luz?

Cuando **aumenta la luz**, la resistencia de la LDR **disminuye** drásticamente:

- Oscuridad total: $R_{LDR} \approx 100 \text{ k}\Omega - 1 \text{ M}\Omega$

- Luz ambiente: $R_{LDR} \approx 10 \text{ k}\Omega$
- Luz directa: $R_{LDR} \approx 1 \text{ k}\Omega$ o menos

6. ¿Qué ventajas ofrece el control inalámbrico mediante Bluetooth?

- **Sin cableado:** Elimina la necesidad de tender cables de control
- **Alcance:** Hasta 10 metros en Bluetooth Clásico
- **Bajo consumo:** Ideal para dispositivos alimentados por batería
- **Sin internet:** No requiere router ni conexión WiFi
- **Universal:** Compatible con cualquier smartphone moderno
- **Bidireccional:** Permite enviar comandos y recibir confirmaciones

7. ¿Qué diferencia existe entre el modo manual y el modo automático?

- **Modo Manual:** El usuario controla directamente el encendido y apagado mediante botones ON/OFF en la aplicación. La decisión es humana.
- **Modo Automático:** El sistema utiliza el sensor LDR para medir la iluminación ambiental. Enciende automáticamente en oscuridad y apaga cuando hay suficiente luz natural, sin intervención del usuario.

8. ¿Qué ventajas tiene Flutter para el desarrollo de aplicaciones móviles?

- **Multiplataforma:** Un solo código para Android, iOS, Web y Desktop
- **Rendimiento nativo:** Compilación AOT a código ARM
- **Hot Reload:** Cambios visibles en milisegundos
- **Widgets ricos:** Material Design 3, Cupertino, personalizables
- **Ecosistema:** Amplia biblioteca de paquetes (pub.dev)
- **Dart:** Lenguaje moderno, fácil de aprender, con null safety

9. ¿Qué ocurre si el relevador no utiliza transistor de activación?

- El pin GPIO se **dañaría permanentemente** por exceso de corriente
- El relevador posiblemente **no se activaría** por voltaje insuficiente
- El pico de voltaje inverso **dañaría el microcontrolador**
- Se perdería el **aislamiento** entre la lógica de control y la carga

10. ¿Qué función cumple el temporizador dentro del sistema?

El temporizador permite **programar el encendido por un tiempo determinado**. El usuario configura minutos/segundos en la app, el sistema enciende el bombillo automáticamente y lo apaga cuando transcurre el tiempo. La app muestra una cuenta regresiva visual. Es útil para:

- Apagado automático para ahorro energético
- Iluminación temporal programada
- Simulación de presencia

Conclusiones

Resultados Obtenidos

1. **Comunicación Bluetooth:** Se implementó exitosamente un servidor Bluetooth SPP en el Pico W utilizando la librería SerialBT, logrando comunicación bidireccional estable con dispositivos Android.
2. **Aplicación Móvil Propia:** Se desarrolló una aplicación completa en Flutter con arquitectura Provider, tres modos de operación (Manual, Timer, Automático) e interfaz Material Design 3.
3. **Control de Potencia:** Se implementó correctamente el circuito de potencia con transistor 2N2222A, diodo de protección 1N4007 y resistencia pull-up de 10 k Ω , logrando controlar una carga de 120V AC de forma segura.
4. **Sensor LDR:** Se integró exitosamente el sensor de iluminación ambiental con lógica corregida para encender en oscuridad y apagar con luz.
5. **Temporizador:** Se implementó conteo regresivo visual en la app mediante Timer.periodic, sincronizado con el temporizador del firmware.

Possibles Mejoras

- Implementar control de intensidad lumínica (dimmer) mediante PWM en el pin GP0 para regular el brillo del bombillo
- Agregar múltiples canales independientes para controlar varias luces simultáneamente
- Integrar control por voz mediante Google Assistant o Alexa
- Desarrollar versión para iOS utilizando la misma base de código Flutter
- Implementar programación horaria con almacenamiento en memoria flash
- Agregar sensor de corriente para medición de consumo energético
- Integrar con plataformas IoT (MQTT, HomeKit, Google Home)

- Añadir autenticación biométrica en la aplicación

Aplicaciones Reales del Proyecto

Este sistema tiene aplicaciones directas en:

- **Domótica:** Automatización de iluminación en hogares inteligentes
- **Oficinas:** Control centralizado de iluminación con sensores de presencia
- **Eficiencia energética:** Apagado automático en espacios desocupados
- **Accesibilidad:** Control remoto para personas con discapacidad motriz
- **Educación:** Plataforma didáctica para enseñanza de IoT y sistemas embebidos
- **Agricultura:** Control de iluminación en invernaderos

Fin del Reporte

Práctica 16 – Control Inteligente de Iluminación Bluetooth
Microcontroladores – Mayo 2026

Referencias

1. Raspberry Pi Foundation. (2024). *Raspberry Pi Pico W Datasheet*. Recuperado de: <https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf>
2. Earle F. Philhower, III. (2024). *Arduino-Pico: Raspberry Pi Pico Arduino Core*. Recuperado de: <https://github.com/earlephilhower/arduino-pico>
3. BlueKitchen GmbH. (2024). *BTstack Manual*. Recuperado de: <https://bluekitchen-gmbh.com/btstack/>
4. Google LLC. (2024). *Flutter Documentation*. Recuperado de: <https://flutter.dev/docs>
5. Bluetooth SIG. (2024). *Bluetooth Core Specification 5.0*. Recuperado de: <https://www.bluetooth.com/specifications/>
6. Arduino. (2024). *Arduino CLI Documentation*. Recuperado de: <https://arduino.github.io/arduino-cli/>